

wanting_35507071_TaskC_and_D

Wanting Echo Zhao

2026-06-05

Task C

```
# Load necessary packages
library(tidyverse)
library(dplyr)
library(tidyr)
library(ggplot2)
library(lubridate)
library(stringr)
library(naniar)
library(data.table)
library(gt)

prop <- read_csv("TaskC_property_victoria.csv")

# overview the data
cat("row: ", nrow(prop), "column:", ncol(prop))
```

```
## row: 418390 column: 12
```

```
# see the type of each attribute
str(prop)
```

```
## spc_tbl_ [418,390 × 12] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ ID : num [1:418390] 1.28e+08 1.23e+08 2.02e+08 2.03e+08 1.40e+08 ...
## $ postcode : num [1:418390] 3095 3074 3978 3753 3197 ...
## $ suburb : chr [1:418390] "Eltham" "Thomastown" "Clyde North" "Beveridge" ...
## $ sold_time : Date[1:418390], format: "2018-07-04" "2016-07-16" ...
## $ sold_price : num [1:418390] 1900000 424000 309000 NA 901000 385000 710000 NA 425
000 650000 ...
## $ address : chr [1:418390] "10 Sweeneys Lane, Eltham, Vic 3095" "2 Banksia Aven
ue, Thomastown, Vic 3074" "74 Yeungroon Boulevard, Clyde North, Vic 3978" "Lot 730, Radi
ata Street, Beveridge, Vic 3753" ...
## $ beds : num [1:418390] 7 3 NA NA 3 1 3 3 1 3 ...
## $ baths : num [1:418390] 4 1 NA NA 1 1 2 2 1 2 ...
## $ parking : num [1:418390] 2 2 NA NA 3 1 2 3 1 1 ...
## $ area : num [1:418390] 4322 659 448 325 638 ...
## $ property_type: chr [1:418390] "house" "house" "residential-land" "residential-lan
d" ...
## $ description : chr [1:418390] "Set in around 4323 m2 of rolling lawns surrounded b
y gardens landscaped by Ben Harris, this is family living on"| __truncated__ "AUCTION TH
IS SATURDAY AT 1PM<br/>Perfectly located in the popular and much sought after Thomastown
West locale "| __truncated__ "Located in the developing St Germain Estate of Clyde North
is this 447 sq mtr block. With a frontage of 14mtrs "| __truncated__ "Fully Fenced<br/>
$10,000 Deposit" ...
## - attr(*, "spec")=
## .. cols(
## .. ID = col_double(),
## .. postcode = col_double(),
## .. suburb = col_character(),
## .. sold_time = col_date(format = ""),
## .. sold_price = col_double(),
## .. address = col_character(),
## .. beds = col_double(),
## .. baths = col_double(),
## .. parking = col_double(),
## .. area = col_double(),
## .. property_type = col_character(),
## .. description = col_character()
## .. )
## - attr(*, "problems")=<externalptr>
```

Question 1: Top 3 suburbs by transaction count + monthly 2022 plot

Mind map

- Find the three suburbs with the highest number of transactions *across all years*.
- Then plot the monthly transaction counts during 2022 for those suburbs.
- If *Toorak* is not already in the top three, include it as a fourth line on the plot.

```
# Compute NA value
miss_var_cumsum(prop)
```

```
## # A tibble: 12 × 3
##   variable      n_miss n_miss_cumsum
##   <chr>         <int>         <int>
## 1 ID             21             21
## 2 postcode       21             42
## 3 suburb         21             63
## 4 sold_time      5218            5281
## 5 sold_price     50425            55706
## 6 address        6158            61864
## 7 beds          19884            81748
## 8 baths          19884           101632
## 9 parking        19884           121516
## 10 area           28           121544
## 11 property_type  21           121565
## 12 description    26           121591
```

```
# There is 21 miss data in `suburb`
```

```
# Find the top 3 suburbs by transaction count
```

```
prop_clean <- prop |>
  filter(!is.na(suburb))
```

```
# Compute the count of transactions of each suburb, then sort descently. slice top 3
```

```
sub_count <- prop_clean |>
  count(suburb, sort = TRUE, name = "transation_count")
```

```
# show top 10
```

```
head(sub_count, 10)
```

```
## # A tibble: 10 × 2
##   suburb      transation_count
##   <chr>         <int>
## 1 Toorak         3818
## 2 Melbourne     3556
## 3 Clyde North   2090
## 4 Mornington    2031
## 5 Prahran       2031
## 6 Preston       2025
## 7 Tarneit       2023
## 8 Officer       2022
## 9 Sunbury       2011
## 10 Truganina     2008
```

```
# Top 1 is Toorak, so there is no need to add it individually for question 1
# extract top 3
top3 <- sub_count$suburb[1:3]
top3
```

```
## [1] "Toorak"      "Melbourne"   "Clyde North"
```

The top three suburbs are **Toorak (3,818)**, **Melbourne (3,556)**, and **Clyde North (2,090)**. Toorak is already #1, so the plot below contains exactly those three lines (no extra Toorak line is needed).

```
# Build a (suburb + month) count table for 2022 only.
```

```
monthly_2022 <- prop_clean |>
  filter(year(sold_time) == 2022,
         suburb %in% top3) |>
  count(suburb, month = month(sold_time), name = "transation_counts") |>
  arrange(suburb, month)
```

```
# Transfer long to wide format
```

```
monthly_2022 |>
  pivot_wider(names_from = month, values_from = transation_counts)
```

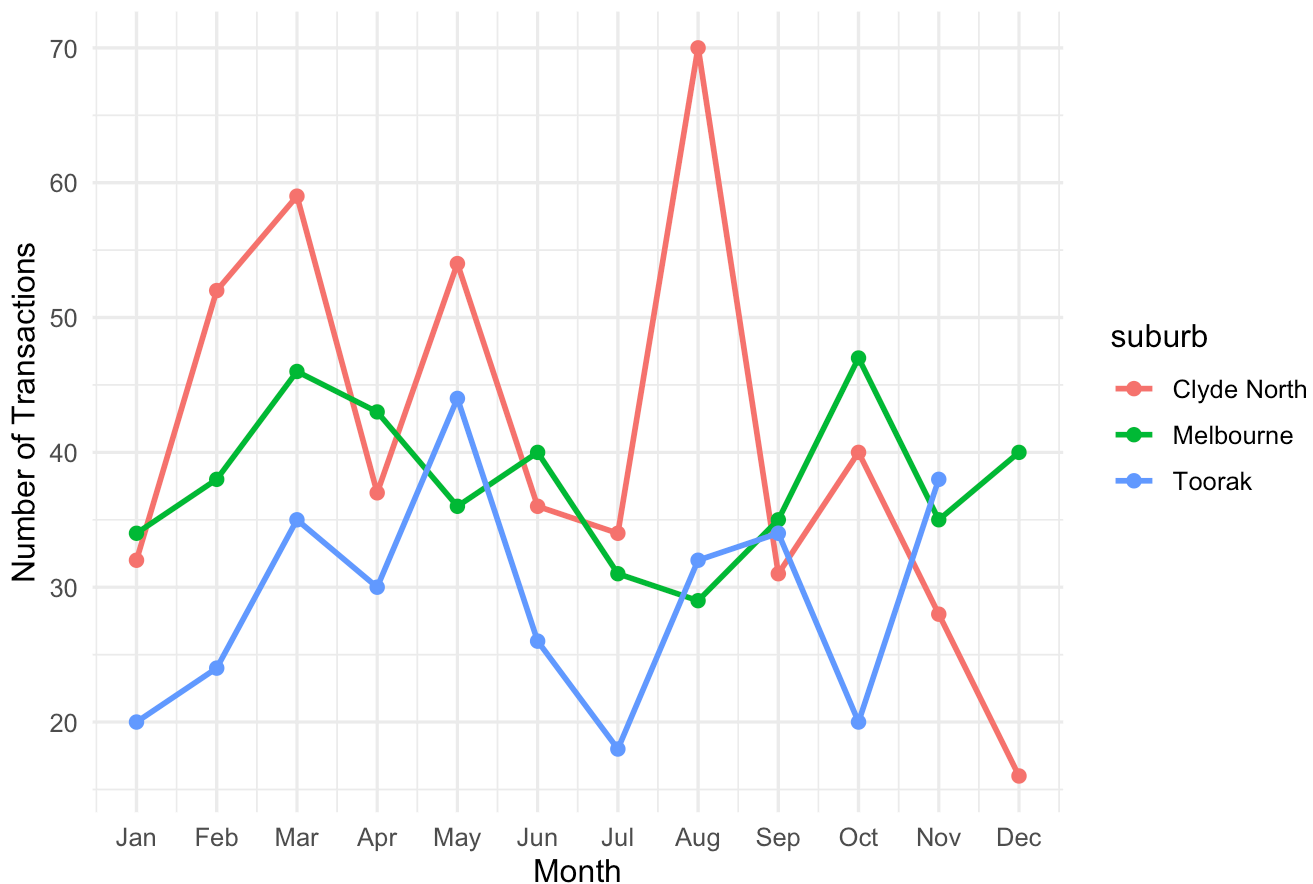
```
## # A tibble: 3 × 13
```

```
##   suburb   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  `10`  `11`  `12`
##   <chr> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 Clyde...    32    52    59    37    54    36    34    70    31    40    28    16
## 2 Melbo...    34    38    46    43    36    40    31    29    35    47    35    40
## 3 Toorak     20    24    35    30    44    26    18    32    34    20    38    NA
```

```
# Plot it
```

```
ggplot(monthly_2022, aes(x = month, y = transation_counts, colour = suburb, group = suburb)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) + # 12 months and show as abbr,
  saving space and looks tidier.
  labs(title = "Monthly property transactions in 2022",
       x = "Month",
       y = "Number of Transactions") +
  theme_minimal(base_size = 12)
```

Monthly property transactions in 2022



Interpretation of plot

Clyde North peaks in August 2022 (70 transactions), the biggest single suburb-month in the chart, before falling sharply at the end of year. Melbourne is the steadiest of the three. Toorak shows the lowest volume of the three and has **zero** December 2022 transactions — likely a data-collection gap rather than a market collapse.

Question 2: The 3 most important keywords in description impacting price

```
library(Matrix) # sparse document-term matrix
library(glmnet) # LASSO

# STEP1: 10% random sample, keeping only rows with a price and a description.
set.seed(123)
samp <- prop |>
  filter(!is.na(sold_price), !is.na(description)) |>
  slice_sample(prop = 0.1)

cat("Sample size:", nrow(samp), "rows\n")
```

```
## Sample size: 36796 rows
```

```
# Having a look at the top 10 row of sample$desription,
# Figure out what kind of messy should we clean
# head(samp$description, 10)

# STEP2: Clean the description text:
clean_text <- samp$description |>
  str_replace_all("<[^>]+>", " ") |> # strip HTML tags like <br/>
  str_to_lower() |> # lower case
  str_replace_all("[^a-z ]+", " ") |> # keep pure letters. Drop all numbers
  str_squish() # collapse multiple space, so later easier to split text.
```

Reference: Week 6 (<https://learning.monash.edu/course/view.php?id=41099§ion=28>) Map-reduce - data table (<https://cran.r-project.org/web/packages/data.table/vignettes/datatable-intro.html>)

```
# STEP3: Tokenise and explode into a long (doc, term) table using data.table
# a 10% sample is still ~37k docs and 28k+ unique terms).
tokens <- str_split(clean_text, " ")
dtm_long <- data.table(doc = rep(seq_along(tokens), lengths(tokens)),
                      term = unlist(tokens))

# STEP4: Remove various stopword

# 1) Grab general English stopwords (Snowball is a reliable, standard lexicon)
general_stopwords <- stopwords::stopwords(language = "en", source = "snowball")

# 2) Define your specific technical and domain-specific noise sequences
html_leftovers <- c("amp", "br", "lt", "gt", "quot", "apos")

real_estate_terms <- c(
  "also", "new", "one", "two", "three", "four", "five",
  "feature", "features", "property", "home", "house", "bed", "bedroom", "bedrooms",
  "bath", "bathroom", "bathrooms", "room", "rooms", "living", "please", "including",
  "call", "inspection", "view", "contact", "auction", "sale"
)

# 3) Combine them into a single, deduplicated vector
stopwords_en <- unique(c(general_stopwords, html_leftovers, real_estate_terms))

# 4) Filter the data.table
dtm_long <- dtm_long[term != "" & nchar(term) >= 3]
dtm_long <- dtm_long[!term %in% stopwords_en]

nrow(dtm_long) # 4029090
```

```
## [1] 4029090
```

```
# STEP5: Vocabulary filter: keep only terms that appear in at least 1% of docs.
# Filter: Rare/unique words make the model unstable; very common words don't help either.
min_df <- ceiling(0.01 * nrow(samp))
keep_terms <- dtm_long[, .(df = uniqueN(doc)), by = term][df >= min_df, term]
dtm_long <- dtm_long[term %in% keep_terms]
dtm_long <- unique(dtm_long, by = c("doc", "term")) # collapse to PRESENCE
cat("Vocabulary size after filtering:", length(keep_terms), "terms\n")
```

```
## Vocabulary size after filtering: 1500 terms
```

```
# STEP6: Build the sparse document-term matrix
term_levels <- sort(unique(dtm_long$term))
dtm <- sparseMatrix(
  i = dtm_long$doc,
  j = match(dtm_long$term, term_levels),
  x = 1,
  dims = c(nrow(samp), length(term_levels)),
  dimnames = list(NULL, term_levels)
)
dim(dtm)
```

```
## [1] 36796 1500
```

```
# 7) Fit LASSO with cross-validated lambda.
y <- log(samp$sold_price)
fit <- cv.glmnet(dtm, y, alpha = 1, nfolds = 5)

# Pull non-zero coefficients at lambda.1se (the more conservative pick:
# the largest lambda whose error is within 1 SE of the minimum).
co <- coef(fit, s = "lambda.1se")
coef_tbl <- tibble(term = rownames(co)[-1], # drop intercept
  coef = as.numeric(co)[-1]) |>
  filter(coef != 0) |>
  arrange(desc(abs(coef)))

# Top 20 for context, then the answer (top 3)
head(coef_tbl, 20)
```

```
## # A tibble: 20 × 2
##   term      coef
##   <chr>    <dbl>
## 1 toorak    0.289
## 2 apartment -0.259
## 3 melton   -0.208
## 4 acres     0.200
## 5 hydronic  0.192
## 6 camberwell 0.182
## 7 marble    0.153
## 8 cellar    0.150
## 9 fireplaces 0.145
## 10 communal -0.136
## 11 doncaster 0.132
## 12 miele     0.130
## 13 stca      0.128
## 14 ofp       0.126
## 15 townhouses 0.121
## 16 victorian 0.120
## 17 butler    0.120
## 18 affordable -0.119
## 19 frankston -0.117
## 20 period    0.111
```

```
top3_keywords <- head(coef_tbl, 3)
top3_keywords
```

```
## # A tibble: 3 × 2
##   term      coef
##   <chr>    <dbl>
## 1 toorak    0.289
## 2 apartment -0.259
## 3 melton   -0.208
```

LASSO (<https://www.r-bloggers.com/2020/05/quick-tutorial-on-lasso-regression-with-example/>)

Answer

The three keywords with the highest coefficients are: toorak (coef: 0.288), apartment (coef: -0.259), melton (coef: -0.208)

- toorak (positive): when “toorak” appears in the description, the predicted price rises.
- apartment (negative): the word “apartment” in the description is associated with a ~26% *lower* predicted price.
- melton (negative): “melton (https://en.wikipedia.org/wiki/Melton,_Victoria)” are associated with a ~21% lower price.

Insights

Some of the top words (*toorak* , *camberwell* , *melton*) are *suburb names*. They function as legitimate description keywords, but they partly proxy for the suburb itself. The other items in the Some keywords in top 20, such as *marble* , *cellar* , *fireplaces* , *hydronic* , *victorian* , *miele* , correspond more directly to property *features* and align human's intuition that luxury fittings drive prices up. This also can be tell by words like *communal* , *affordable* with negative coefficient, which indicating that the presence of these words suggests lower property prices.

Question 3: Correlation of price and land size by (top-3 suburb × property type)

Mind map

- Get each suburb among the top 3 suburbs identified in Q1 - Extract specific columns
- sold_price and area - sold_price remove NA - area remove minus value and NA. so I filter on area > 0 (this drops both NA and -1).
- for each suburb and for each property type suggest group .

Because small sample sizes (like only 2 observations) artificially force a perfect correlation of, flagging low n_obs helps readers know which results are reliable and which are misleading.

obs: www.wisdomlib.org. (2025, August 18). Number of observations: Significance and symbolism. <https://www.wisdomlib.org/concept/number-of-observations> (<https://www.wisdomlib.org/concept/number-of-observations>)

```
top3_suburbs <- c("Toorak", "Melbourne", "Clyde North")
types_wanted <- c("house", "unit", "townhouse", "apartment")

prop_q3 <- prop |>
  filter(suburb %in% top3_suburbs,
         property_type %in% types_wanted,
         !is.na(sold_price),
         !is.na(area), area > 0)

# How many suburb and property
prop_q3 |>
  count(suburb, property_type) |>
  pivot_wider(names_from = property_type, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 × 5
##   suburb      house townhouse unit apartment
##   <chr>      <int>      <int> <int>      <int>
## 1 Clyde North   808         43      1          0
## 2 Melbourne     0          0      2        139
## 3 Toorak       334          9      0         11
```

```
# Compute the correlation per group
cor_tbl <- prop_q3 |>
  group_by(suburb, property_type) |>
  summarise(n_obs = n(),
            correlation = if(n() >= 2) cor(sold_price, area) else NA_real_,
            .groups = "drop") |>
  filter(!is.na(correlation)) |>
  arrange(suburb, property_type)

cor_tbl
```

```
## # A tibble: 7 × 4
##   suburb      property_type n_obs correlation
##   <chr>      <chr>      <int>      <dbl>
## 1 Clyde North house         808      0.323
## 2 Clyde North townhouse      43      0.483
## 3 Melbourne apartment     139      0.0761
## 4 Melbourne unit           2        1
## 5 Toorak apartment       11      0.899
## 6 Toorak house         334      0.706
## 7 Toorak townhouse        9      0.391
```

Findings

Cells with reasonable sample sizes(n_{obs}):

- **Toorak / house** ($n = 334$, $r \approx 0.71$): strong positive — in this suburb, bigger land translates into substantially higher prices.
- **Clyde North / house** ($n = 808$, $r \approx 0.32$): moderate positive — outer-suburb new builds, where land matters but standardised house specs flatten the relationship somewhat.
- **Melbourne / apartment** ($n = 139$, $r \approx 0.08$): essentially no relationship — for inner-city apartments, location, floor, and finishes dominate; the recorded land area is barely informative.

Cells with very few observations — **Melbourne / unit** ($n = 2$), **Toorak / townhouse** ($n = 9$), **Toorak / apartment** ($n = 11$) are kept for completeness but should be read with caution; $r = 1$ for Melbourne / unit is a mathematical artefact of having only two data points, not a meaningful finding.

Note: There are no apartment recorded in Clyde North, 0 house and townhouse in Melbourn, and non unit in Toorak.

Question 4: Top 5 capital gains (resold within 5 years)

Mind Map

- Which properties means lock property_type
- highest price increases , as far as I understand, there are two ways to measure. One is to compare the price increase by the gap between first sold_price and its maximum value. Second is to compare popular.
- exclude properties where the time between the first and last sale exceeds five years. means mutate a new column, computing the duration, filter ones exceed 5.
- List the top five: Sort and
- address, capital gain, and the duration between the first and last sale.:select these columns.

Challenges and Solutions

1. Only sold_time , the ID is transaction number instead of identity of each property. property with same postcode and address should be same one, and they can be encode with a new "ID". The reason of why using "address + postcode" as key, even address include postcode is to uniquely identify properties safely, in case there are typo.
2. Extreme value handling. In order to avoid impaction by outliers. control the range of sold_price

```
head(sort(prop$sold_price),5)
```

```
## [1] 1 1 1 1 270
```

```
tail(sort(prop$sold_price),5)
```

```
## [1] 320000000 80000088 80000088 320000000 1010000000
```

Using chunk above you can notice the minimum price would be 1 and the top two (\$1.01B and \$320M) are implausible. I therefore restrict to prices in the range $\$50,000 \leq \text{sold_price} \leq \$50,000,000$ before computing capital gains. This keeps the genuine premium Toorak sales (up to $\approx \$32\text{M}$) while removing the obvious typos.

```

# 1) Filter to rows with valid address, plausible price, and valid date.
ok <- prop |>
  filter(!is.na(address), address != "",
         !is.na(sold_price), sold_price >= 50000, sold_price <= 50e6,
         !is.na(sold_time))

# 2) Synthesis filtered properties using data.table.

ok_dt <- as.data.table(ok)

per_property <- ok_dt[, .(
  n_sales      = .N,
  first_date   = min(sold_time),    # column first_date
  last_date    = max(sold_time),    # column last_date
  first_price  = sold_price[which.min(sold_time)], # column first_price
  last_price   = sold_price[which.max(sold_time)]  # column last_price
), by = .(address, postcode)      # double insurance - filter the same property
][n_sales >= 2 & last_date > first_date] # at least two different-day sales

# 3) Derive capital gain and duration in years
per_property$gain <- per_property$last_price - per_property$first_price
per_property$duration_years <- as.numeric(per_property$last_date - per_property$first_date) / 365

# 4) Apply the ≤ 5-year filter and pick the top 5

top5_q4 <- per_property |>
  as_tibble() |> # output should be a table
  filter(duration_years <= 5) |>
  arrange(desc(gain)) |>
  slice_head( n = 5 ) |>
  mutate(duration_years = round(duration_years,2)) |>
  select(address, gain, duration_years,
         first_date, first_price, last_date, last_price)

# 5) See the below
library(knitr)
library(kableExtra)

# Generate table and save directly to file
kable(top5_q4, format = "html") |>
  kable_styling() |>
  save_kable("Top5_gain_property.png")

```

Answer

Table (Top5_gain_property.png)

Question 5: Properties that could not realistically exist

Mind map

- Removing negative values from area.

```
# Drop negative area values per the spec. (NA area is allowed to stay –  
# the instruction only says to remove *negative* values.)  
prop_q5 <- prop |> filter(is.na(area) | area >= 0)  
cat("Rows after removing negative area:", nrow(prop_q5), "\n")
```

```
## Rows after removing negative area: 191168
```

Implausible senarios

```
# 1) Tiny area for a multi-bedroom  
row1 <- prop_q5 |>  
  filter(area>=1, area <=3, beds >= 3) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  head(1)
```

```
# 2) Implausibly large parking space  
row2 <- prop_q5 |>  
  filter(parking >= 100) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  arrange(desc(parking)) |>  
  head(1)
```

```
# 3) Implausibly large bedroom counts in a single house  
row3 <- prop_q5 |>  
  filter(beds >= 20) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  arrange(desc(beds)) |>  
  head(1)
```

```
# 4) Implausibly large bathroom counts in a single house  
row4 <- prop_q5 |>  
  filter(baths >= 18, area <= 5) |>  
  select(ID, address, beds, baths, parking, area, property_type) |>  
  arrange(desc(baths)) |>  
  head(1)
```

```
# 5) Implausibly large bathroom counts more than bedroom
row5 <- prop_q5 |>
  filter(beds == 0, baths >= 3) |>
  select(ID, address, beds, baths, parking, area, property_type) |>
  arrange(desc(baths)) |>
  head(1)
```

```
implusiabable_table <- bind_rows(row1, row2, row3, row4, row5)

# use `implusiabable_table` to generate a final table like assignment requires
summary_table <- implusiabable_table |>
  mutate(
    Feature = c("area", "parking", "beds", "baths", "beds + baths"),
    Reason = c(
      "A house on ~1 m2 of land cannot physically hold 3+ bedrooms.",
      "11,951 parking spaces for a suburban house (next-highest in the data is 200).",
      "30 bedrooms in a single house; the address field is also corrupted.",
      "20 bathrooms recorded on a property of only ~1 m2 of land.",
      "0 bedrooms but 3+ bathrooms - impossible for a residential house."
    )
  ) |>
  select(Feature, Reason, Address = address, `Property ID` = ID)

knitr::kable(summary_table) # renders as a proper table in the knit.
```

Feature	Reason	Address	Property ID
area	A house on ~1 m2 of land cannot physically hold 3+ bedrooms.	37 Beatties Road, Koo Wee Rup, Vic 3981	107405202
parking	11,951 parking spaces for a suburban house (next-highest in the data is 200).	50 Beverley Road, McCrae, Vic 3938	126712114
beds	30 bedrooms in a single house; the address field is also corrupted.	83 Bedrooms, 34 Bathrooms, Burwood, Vic 3125	118443975
baths	20 bathrooms recorded on a property of only ~1 m2 of land.	18 Park Road, Warburton, Vic 3799	124748374
beds + baths	0 bedrooms but 3+ bathrooms - impossible for a residential house.	38 Fitzwilliam Drive, Doreen, Vic 3754	132715294

Question 6: Predicted July 2026 price for Chris's house in 6 suburbs

Mind map

- Near shopping center and primary school
- 4 bedrooms and 2 bathroom
- in six suburbs—Mulgrave, Vermont South, Doncaster East, Rowville, Glen Waverley, and Wheelers Hill.
- output: price for July 2026, Note: the sold time range is ["2010-01-01", "2023-04-14"]. Hence need predict 3 years later.

```
target_suburbs <- c("Mulgrave", "Vermont South", "Doncaster East", "Rowville", "Glen Waverley", "Wheelers Hill")
```

```
# filter houses in these 6 suburbs
```

```
prop_q6 <- prop |>
  filter(property_type == "house",
         suburb %in% target_suburbs,
         !is.na(sold_price),
         !is.na(area), area>0,
         !is.na(beds), !is.na(baths),
         !is.na(sold_time), !is.na(description))
```

```
prop_q6 <- prop_q6 |>
  mutate(desc_lower = str_to_lower(description),
         has_renovated = str_detect(desc_lower, "renovat"),
         has_shopping = str_detect(desc_lower, "shopping"),
         has_primary_school = str_detect(desc_lower, "primary school"),
         # Continuous time variable so the model can learn a price trend over
         # the years and project it forward to a future date. The (month-1)/12
         # term pins the prediction to a specific month (July 2026 = 2026.5), not
         # just "sometime in 2026".
         year_num = year(sold_time) + (month(sold_time) - 1) / 12)
```

```
# Cap area at the 99th percentile to limit a few giant blocks
```

```
area_cap <- quantile(prop$area, 0.99, na.rm = T)
```

```
# quantile lets the data tell you where the boundary is. It automatically scales to your
dataset, whether you are measuring tiny apartments or massive commercial plots.
```

```
# Using the 99th percentile guarantees that you only flag exactly the top 1% of your absolute most extreme data points.
```

```
prop_q6 <- prop_q6 |>
  mutate(area = pmin(area, area_cap))
```

```
# pmin() (Element-wise Minimum): Looks at one row at a time, makes a choice for that specific row, and moves to the next. that is it compares the original area value of each individual row against your area_cap value, and it keeps whichever number is smaller.
```

```
cat("Training rows per suburb:\n")
```

```
## Training rows per suburb:
```

```
prop_q6 |> count(suburb)
```

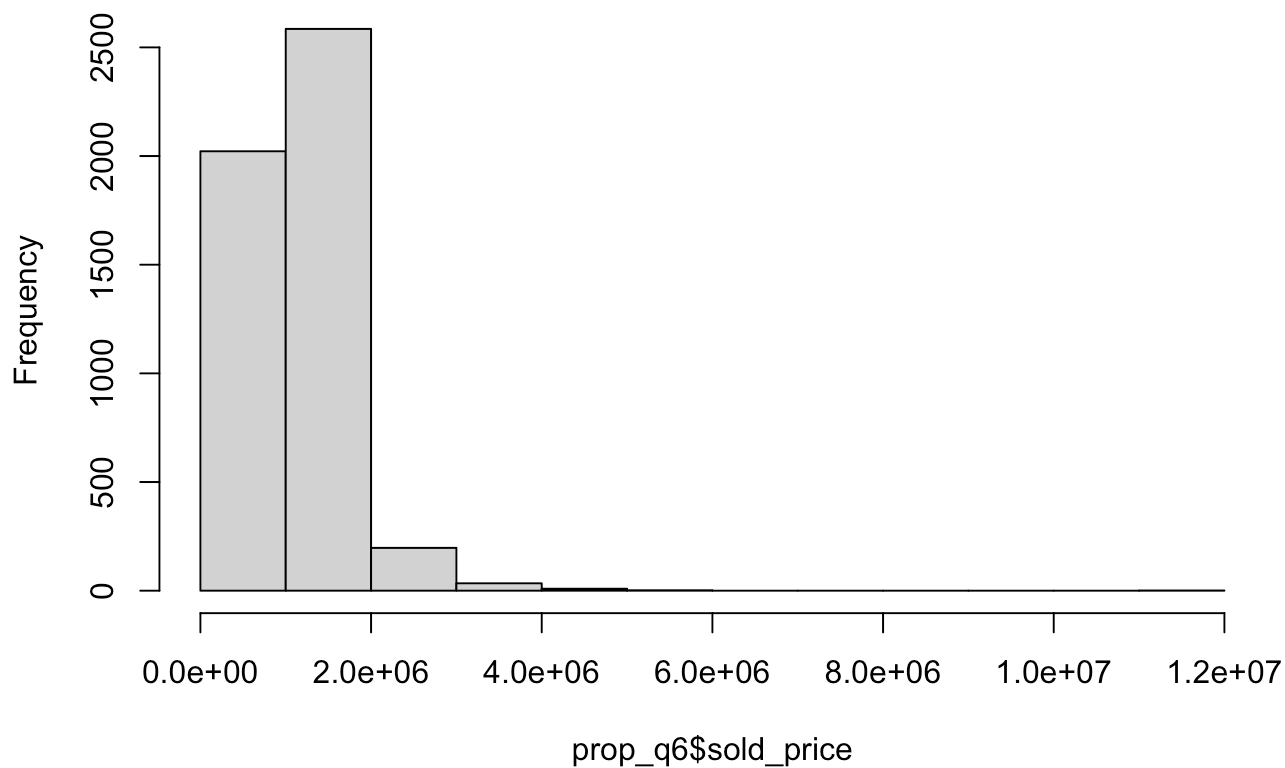
```
## # A tibble: 6 × 2
##   suburb      n
##   <chr>    <int>
## 1 Doncaster East    860
## 2 Glen Waverley   1077
## 3 Mulgrave         747
## 4 Rowville         870
## 5 Vermont South   460
## 6 Wheelers Hill    836
```

```
# Check skewness for choose a better Y(sold_price)
```

```
# Plot raw price
```

```
hist(prop_q6$sold_price, main = "Raw Price (Usually Skewed)")
```

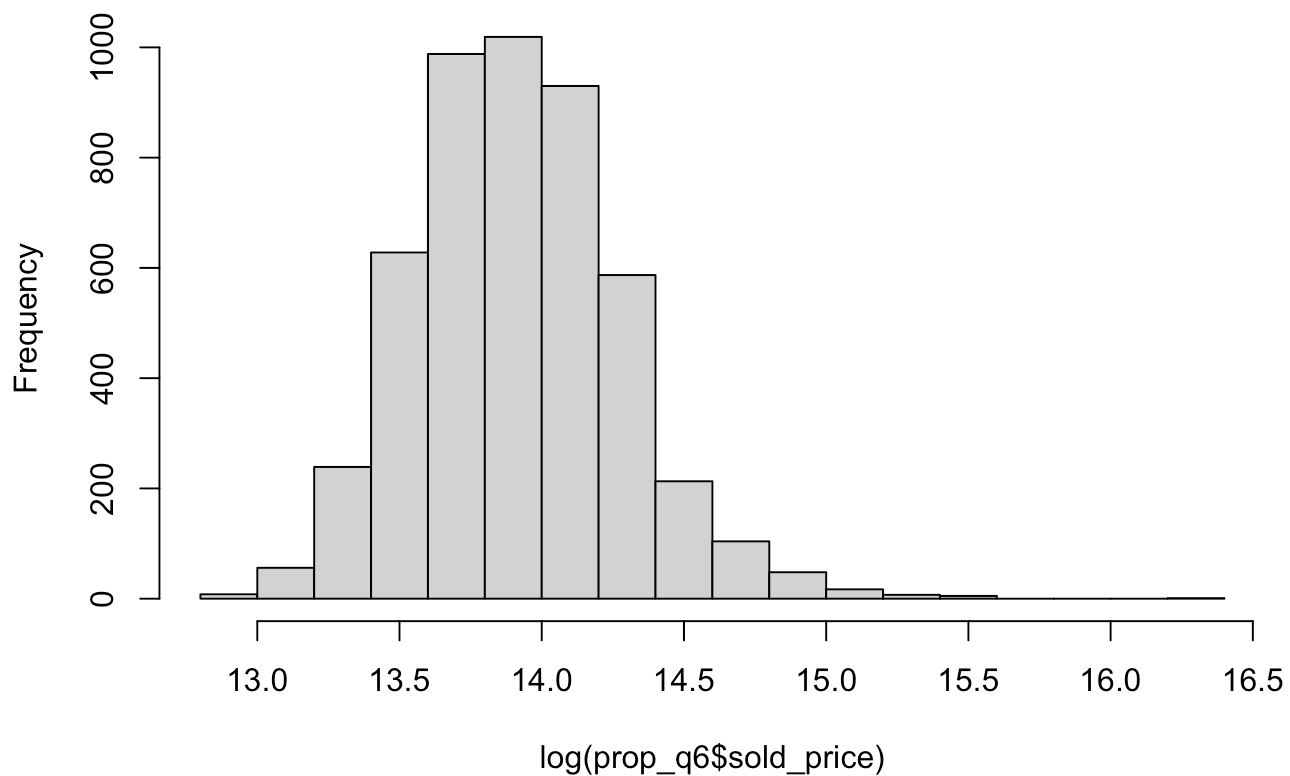
Raw Price (Usually Skewed)



```
# Plot log price
```

```
hist(log(prop_q6$sold_price), main = "Log Price (Usually Bell-Shaped)")
```


Log Price (Usually Bell-Shaped)



Raw sold_price is heavy right skew, so apply `log(sold_price)` in regression model.

```
model_q6 <- lm(log(sold_price) ~ suburb + beds + baths + parking + area +  
               year_num + has_renovated + has_shopping + has_primary_school,  
               data = prop_q6)  
  
summary(model_q6)
```

```
##
## Call:
## lm(formula = log(sold_price) ~ suburb + beds + baths + parking +
##      area + year_num + has_renovated + has_shopping + has_primary_school,
##      data = prop_q6)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.75131 -0.12279 -0.01064  0.10845  1.15475
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.038e+02  1.826e+00 -56.836 < 2e-16 ***
## suburbGlen Waverley    1.191e-01  8.883e-03  13.410 < 2e-16 ***
## suburbMulgrave    -2.784e-01  9.926e-03 -28.047 < 2e-16 ***
## suburbRowville    -3.884e-01  9.440e-03 -41.148 < 2e-16 ***
## suburbVermont South -1.121e-01  1.122e-02  -9.987 < 2e-16 ***
## suburbWheelers Hill -1.016e-01  9.510e-03 -10.680 < 2e-16 ***
## beds              4.998e-02  4.461e-03  11.203 < 2e-16 ***
## baths             1.136e-01  4.861e-03  23.368 < 2e-16 ***
## parking           8.296e-03  3.283e-03   2.527  0.01154 *
## area              3.660e-04  1.149e-05  31.864 < 2e-16 ***
## year_num          5.804e-02  9.061e-04  64.060 < 2e-16 ***
## has_renovatedTRUE  -1.827e-02  6.315e-03  -2.892  0.00384 **
## has_shoppingTRUE   -1.493e-02  6.297e-03  -2.371  0.01778 *
## has_primary_schoolTRUE -9.194e-03  6.116e-03  -1.503  0.13282
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1918 on 4836 degrees of freedom
## Multiple R-squared:  0.7172, Adjusted R-squared:  0.7165
## F-statistic: 943.5 on 13 and 4836 DF, p-value: < 2.2e-16
```

```
# Quick evaluate the model fitness
```

```
cat("Adjusted R-squared:", round(summary(model_q6)$adj.r.squared, 2), "\n")
```

```
## Adjusted R-squared: 0.72
```

```
cat("Training RMSE:", round(sqrt(mean(residuals(model_q6)^2)),2))
```

```
## Training RMSE: 0.19
```

Mind map

Imaging I am a staff of a real estate company, and Chris is my customer. He tells me his expectation, if I was him, I would like to see a table sheet. “suburb, predict price, the fluctuation”, so that I can plan my financial budget.

Currently, I have built a good fit model, the questions are parking and area not be mentioned by Chris. When predict, it requires input specific value for each attribute. Hence take assumption firstly, filling with median value of them, which won't be distorted by outliers like mean.

```
# Plug in suburb's median value of area and parking into prediction model
# In order to remove impact of outliers, using Median instead of Mean.
typical <- prop_q6 |>
  filter( beds >= 4, baths >= 2) |>
  group_by(suburb) |>
  summarise(area = median(area),
            parking = median(parking),
            n_match = n(), # number of house meet these requirements
            .groups = "drop" # always remember to keep your data clean
  )

# Chris requirements
chris_input <- typical |>
  mutate(
    beds = 4, baths = 2,
    year_num = 2026 + 7/12, # July 2026, convert it to continue value
    has_renovated = T,
    has_shopping = T,
    has_primary_school = T)

# Prediction
log_pred <- predict(model_q6, chris_input, interval = "prediction", level = 0.8) # Assume Chris will consider +/- 20% fluctuation.

# Eventually output
answers <- chris_input |>
  mutate(predict_price = round(exp(log_pred[, "fit"])),
        predict_price_lower_80 = round(exp(log_pred[, "lwr"])),
        predict_price_higher_80 = round(exp(log_pred[, "upr"]))) |>
  arrange(desc(predict_price)) |> # assume Chris want to filter from expensive to cheap.
  select(suburb, predict_price, predict_price_lower_80, predict_price_higher_80, area, parking, n_match)
```

Answer

The 80% prediction intervals span roughly $\pm 20\%$, which is realistic of houses within a single suburb. The ordering matches the well-known pricing hierarchy of these SE Melbourne suburbs — Glen Waverley consistently commands a premium because of the high-demand school zone, while Mulgrave and Rowville sit at the more affordable end. >learn more details (<https://share.google/aimode/ID7TZSbJ77HUr5Bov>)

Task D

Question 1

Mind map

- Find differences between faculties and sexes: extract these columns and convert to factors.
- Extract them to individual datasets for further testing.
- Compare differences between groups using t-test and ANOVA.

```
# Load data and convert relevant columns to factors
independent_data <- read.csv("TaskD_Dataset/forum_ind_train.csv", stringsAsFactors = FALSE)
independent_data$demographic_sex <- factor(independent_data$demographic_sex)
independent_data$unit_faculty <- factor(independent_data$unit_faculty)

# Preview the data
head(independent_data$demographic_sex)
```

```
## [1] F M M M F M
## Levels: F M X
```

```
head(independent_data$unit_faculty)
```

```
## [1] Faculty of Art Des & Architecture Faculty of Art Des & Architecture
## [3] Faculty of Art Des & Architecture Faculty of Art Des & Architecture
## [5] Faculty of Art Des & Architecture Faculty of Art Des & Architecture
## 10 Levels: Faculty of Art Des & Architecture ... Faculty of Science
```

```
dplyr::count(independent_data, demographic_sex)
```

```
##   demographic_sex    n
## 1                F 1540
## 2                M 1043
## 3                X    2
## 4               <NA>    5
```

```
# Identify LIWC variables (all except ID and categoricals)
liwc_vars <- setdiff(names(independent_data), c("Unique_ID", "unit_faculty", "demographic_sex"))
```

Compare by Sex

```
# Remove rows with ambiguous or missing sex values
sex_data <- independent_data[independent_data$demographic_sex %in% c("F", "M"), ]
sex_data$demographic_sex <- droplevels(sex_data$demographic_sex)

# Function to run t-test for each LIWC variable by sex
sex_ttest_results <- lapply(liwc_vars, function(v) {
  tt <- t.test(sex_data[[v]] ~ sex_data$demographic_sex)
  data.frame(
    variable = v,
    mean_F = tt$estimate[1],
    mean_M = tt$estimate[2],
    diff = tt$estimate[1] - tt$estimate[2],
    p_value = tt$p.value
  )
})
sex_results <- do.call(rbind, sex_ttest_results) # combine two data frames
sex_results$p_adj <- p.adjust(sex_results$p_value, method = "BH") # Adjust for multiple
comparisons to control the false discovery rate or family-wise error rate. Benjamini-Hoc
hberg, controls FDR
sex_results <- sex_results[order(sex_results$p_adj), ]
head(sex_results, 10)
```

##		variable	mean_F	mean_M	diff	p_value
##	mean in group F7	Dic	80.4855909	77.94405561	2.5415353	1.396408e-05
##	mean in group F36	family	0.1852208	0.06485139	0.1203694	4.756860e-05
##	mean in group F35	social	8.9888961	7.83867689	1.1502192	9.892704e-05
##	mean in group F87	Exclam	1.3851623	0.43369128	0.9514711	2.275302e-04
##	mean in group F27	number	2.5308766	3.46762224	-0.9367456	4.274295e-04
##	mean in group F51	bio	1.3518701	1.00059444	0.3512757	8.316909e-04
##	mean in group F2	Clout	60.0347468	56.65326942	3.3814773	1.279430e-03
##	mean in group F8	function.	46.7163701	45.16945350	1.5469166	1.540299e-03
##	mean in group F45	certain	1.3326948	1.08024928	0.2524455	3.190315e-03
##	mean in group F	WC	122.0798701	105.53595398	16.5439162	6.020217e-03
##		p_adj				
##	mean in group F7		0.001298659			
##	mean in group F36		0.002211940			
##	mean in group F35		0.003066738			
##	mean in group F87		0.005290078			
##	mean in group F27		0.007950190			
##	mean in group F51		0.012891208			
##	mean in group F2		0.016998148			
##	mean in group F8		0.017905977			
##	mean in group F45		0.032966593			
##	mean in group F		0.040906195			

Compare by Faculty

```
# Remove rows with missing faculty
faculty_data <- independent_data[!is.na(independent_data$unit_faculty), ]
faculty_data$unit_faculty <- droplevels(faculty_data$unit_faculty)

# Function to run ANOVA for each LIWC variable by faculty
faculty_anova_results <- lapply(liwc_vars, function(v) {
  fit <- aov(faculty_data[[v]] ~ faculty_data$unit_faculty)
  p <- summary(fit)[[1]][["Pr(>F)"]][1]
  data.frame(variable = v, p_value = p)
})
fac_results <- do.call(rbind, faculty_anova_results)
fac_results$p_adj <- p.adjust(fac_results$p_value, method = "BH")
fac_results <- fac_results[order(fac_results$p_adj), ]
head(fac_results, 10)
```

```
##      variable      p_value      p_adj
## 73    money 4.777871e-152 4.443420e-150
## 54   health 6.690237e-122 3.110960e-120
## 52     bio 3.352807e-90  1.039370e-88
## 70    work 1.612131e-63  3.748204e-62
## 8      Dic 2.125717e-33  3.953834e-32
## 7   Sixltr 3.182575e-33  4.932992e-32
## 28   number 1.476799e-32  1.962033e-31
## 36   social 1.967062e-32  2.286709e-31
## 1      WC 3.328604e-26  3.439558e-25
## 16    they 2.161596e-24  1.827531e-23
```

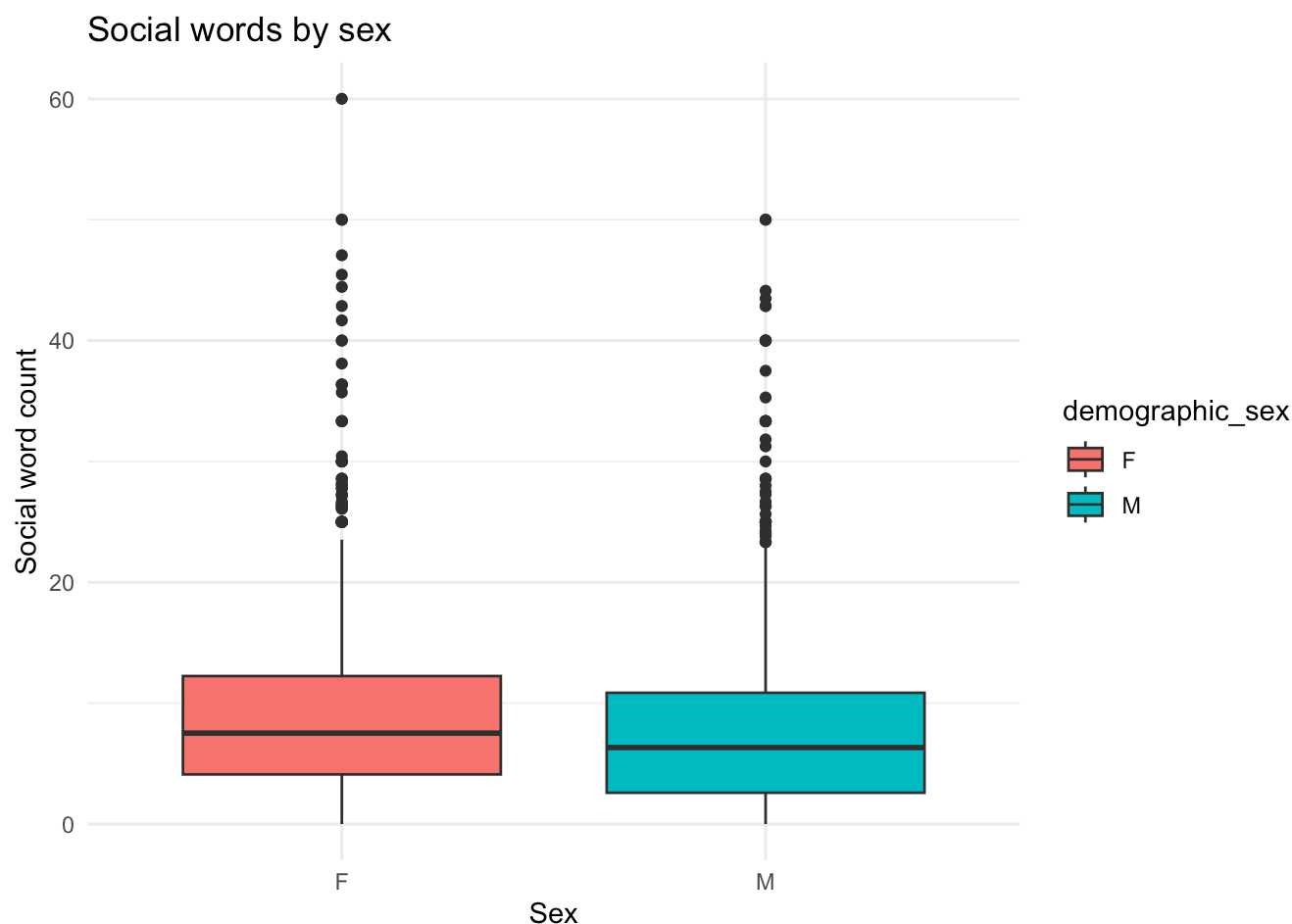
Group Means for Standout Variable

```
# Example: mean of 'money' variable by faculty
tapply(faculty_data$money, faculty_data$unit_faculty, mean)
```

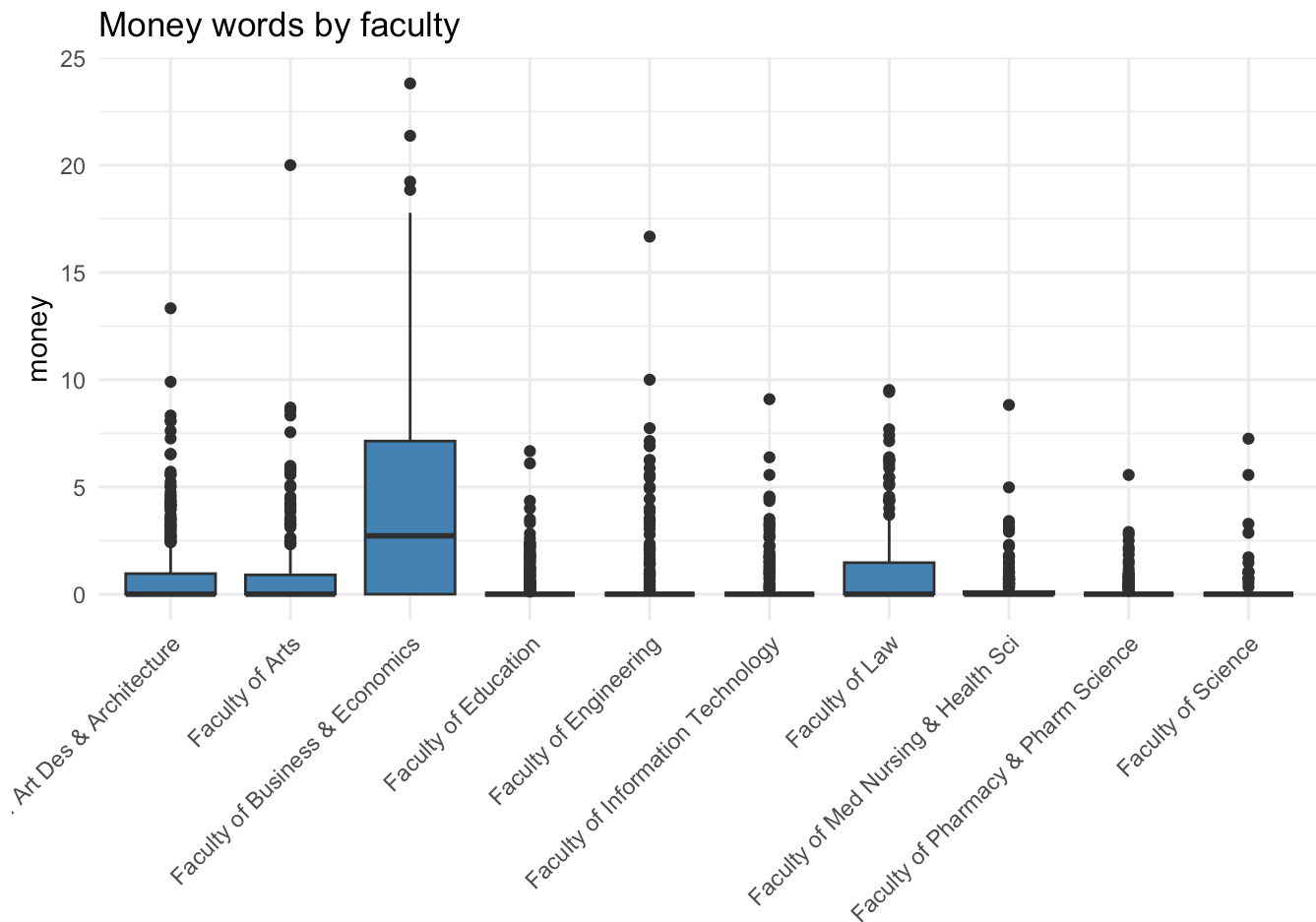
```
##      Faculty of Art Des & Architecture      Faculty of Arts
##                                0.8529041      0.8989895
##      Faculty of Business & Economics      Faculty of Education
##                                4.3795185      0.3338516
##      Faculty of Engineering      Faculty of Information Technology
##                                0.5569286      0.3256679
##      Faculty of Law      Faculty of Med Nursing & Health Sci
##                                1.0695370      0.3118537
##      Faculty of Pharmacy & Pharm Science      Faculty of Science
##                                0.2048485      0.1339320
```

Plots

```
library(ggplot2)
# Boxplot: Social words by sex
ggplot(sex_data, aes(demographic_sex, social, fill = demographic_sex)) +
  geom_boxplot() +
  labs(title = "Social words by sex", x = "Sex", y = "Social word count") +
  theme_minimal()
```



```
# Boxplot: Money words by faculty
ggplot(faculty_data, aes(unit_faculty, money)) +
  geom_boxplot(fill = "steelblue") +
  labs(title = "Money words by faculty", x = NULL, y = "money") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Sex:

15 of 93 variables differ significantly (adjusted $p < 0.05$). The biggest gaps: women's posts score higher on social (8.99 vs 7.84), family (0.19 vs 0.06), Exclam (1.39 vs 0.43), Clout, and are longer overall (WC 122 vs 106 words); men score higher on number (3.47 vs 2.53). These line up with well-documented LIWC gender patterns — more affiliative/social language from women, more numeric language from men.

Faculty:

78 of 93 variables differ — faculty effects are far stronger than sex effects. The standouts are topic words: money is dominated by Business & Economics (mean 4.38 vs ~0.1-0.9 everywhere else), and health/bio/work follow the same disciplinary logic.

Differences between faculties are much larger and more pervasive than differences between sexes, and the strongest faculty differences are content/topic words that reflect what each discipline talks about.

Question 2

The ind and dep files are joined on Unique_ID to form the modelling table, and a single cleaning function is applied to both train and validation so they are treated identically. I evaluate three classifiers covered in the unit — a decision tree, logistic regression and naive Bayes — each trained on all independent variables. Performance is measured with accuracy, precision, recall and F1, treating Content-relevant as the positive class so that F1 is consistent with the metric used for Q4 and Q5.


```

# Load required libraries
library(rpart)
library(e1071)
library(MLmetrics)

# Helper function: join independent and dependent data by Unique_ID
read_join <- function(ind_file, dep_file) {
  x <- read.csv(ind_file, stringsAsFactors = FALSE)
  y <- read.csv(dep_file, stringsAsFactors = FALSE)
  merge(x, y, by = "Unique_ID")
}

# Helper function: clean dataset (remove duplicates, handle NAs, set factors)
clean <- function(df) {
  df <- df[!duplicated(df$Unique_ID), ] # Remove duplicate IDs
  num <- setdiff(names(df), c("Unique_ID", "unit_faculty", "demographic_sex", "label"))
  df <- df[rowSums(is.na(df[num])) < length(num), ] # Remove all-NA rows
  df <- df[!is.na(df$label), ] # Remove rows with missing label
  df$unit_faculty <- ifelse(is.na(df$unit_faculty), "Unknown", df$unit_faculty)
  df$demographic_sex <- ifelse(df$demographic_sex %in% c("F", "M"), df$demographic_sex,
"Unknown")
  df$unit_faculty <- factor(df$unit_faculty)
  df$demographic_sex <- factor(df$demographic_sex)
  df$label <- factor(df$label, levels = c("Content-irrelevant", "Content-relevant"))
  df
}

# Prepare training and validation sets
train <- clean(read_join("TaskD_Dataset/forum_ind_train.csv", "TaskD_Dataset/forum_dep_t
rain.csv"))
val <- clean(read_join("TaskD_Dataset/forum_ind_validation.csv", "TaskD_Dataset/forum_de
p_validation.csv"))
val$unit_faculty <- factor(val$unit_faculty, levels = levels(train$unit_faculty))
val$demographic_sex <- factor(val$demographic_sex, levels = levels(train$demographic_se
x))

# Define predictors and formula
predictors <- setdiff(names(train), c("Unique_ID", "label"))
form <- as.formula(paste("label ~", paste(predictors, collapse = " + ")))

# Helper function: evaluate model metrics
eval_metrics <- function(actual, predicted, positive = "Content-relevant") {
  data.frame(
    accuracy = Accuracy(y_pred = predicted, y_true = actual),
    precision = Precision(y_true = actual, y_pred = predicted, positive = positive),
    recall = Recall(y_true = actual, y_pred = predicted, positive = positive),
    f1 = F1_Score(y_true = actual, y_pred = predicted, positive = positive)
  )
}

# Fit and predict: Decision Tree
fit_tree <- rpart(form, data = train, method = "class")

```

```

pred_tree <- predict(fit_tree, val, type = "class")

# Fit and predict: Logistic Regression
fit_glm <- glm(form, data = train, family = binomial)
pred_glm <- ifelse(predict(fit_glm, val, type = "response") > 0.5, "Content-relevant",
"Content-irrelevant")

# Fit and predict: Naive Bayes
fit_nb <- naiveBayes(form, data = train)
pred_nb <- predict(fit_nb, val)

# Combine results for easy comparison
results <- rbind(
  cbind(model = "Decision tree", eval_metrics(val$label, pred_tree)),
  cbind(model = "Logistic regression", eval_metrics(val$label, pred_glm)),
  cbind(model = "Naive Bayes", eval_metrics(val$label, pred_nb))
)
results

```

##	model	accuracy	precision	recall	f1
## 1	Decision tree	0.8630631	0.8971722	0.9064935	0.9018088
## 2	Logistic regression	0.8810811	0.8880779	0.9480519	0.9170854
## 3	Naive Bayes	0.8090090	0.8163265	0.9350649	0.8716707

Logistic regression performs best on the validation set ($F1 = 0.917$, $accuracy = 0.881$), ahead of the **decision tree** ($F1 = 0.902$) and **naive Bayes** ($F1 = 0.872$). Logistic regression is chosen as Model 1. **Naive Bayes** has the weakest precision, likely due to its strong independence assumption being violated by correlated LIWC variables.

Note: Fitting logistic regression on all 95 predictors triggers a “fitted probabilities numerically 0 or 1” warning, indicating quasi-separation and a risk of overfitting. This motivates the variable-selection and alternative-model experiments in Q4.

Question 3

```

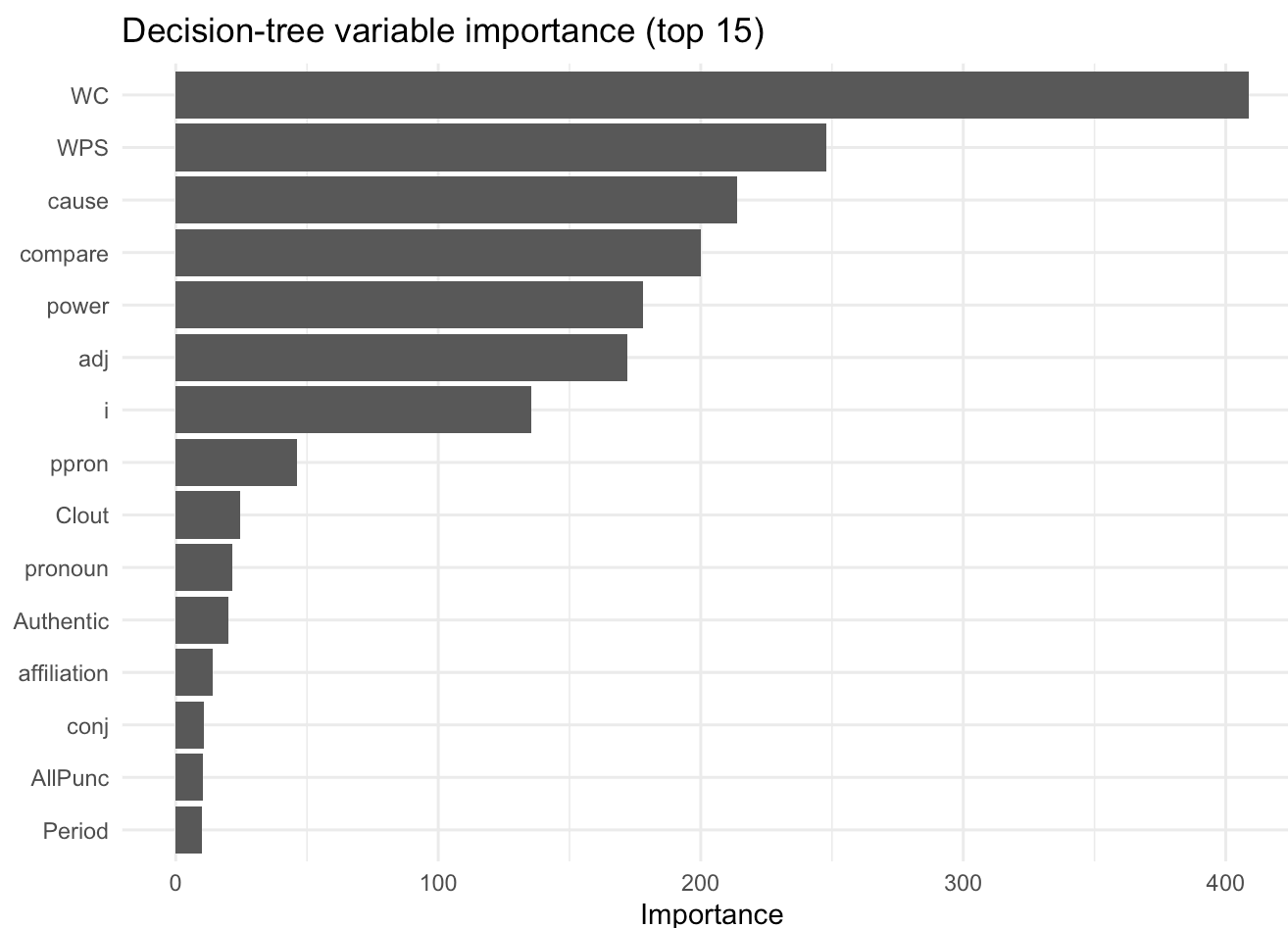
library(rpart)
library(ggplot2)

# Method 1: Decision-tree variable importance
fit_tree <- rpart(form, data = train, method = "class")
imp <- sort(fit_tree$variable.importance, decreasing = TRUE)
head(imp, 15) # Show top 15 important variables

```

##	WC	WPS	cause	compare	power	adj
##	408.638714	247.797265	213.886009	199.974974	177.949758	172.006446
##	i	ppron	Clout	pronoun	Authentic	affiliation
##	135.259027	46.347070	24.365380	21.691697	19.860817	14.169445
##	conj	AllPunc	Period			
##	10.616691	10.218565	9.820439			

```
# Plot top 15 important variables
imp_df <- data.frame(variable = names(imp), importance = as.numeric(imp))[1:15, ]
imp_df$variable <- factor(imp_df$variable, levels = rev(imp_df$variable))
ggplot(imp_df, aes(variable, importance)) +
  geom_col() +
  coord_flip() +
  labs(title = "Decision-tree variable importance (top 15)", x = NULL, y = "Importance")
+
  theme_minimal()
```



```
# Method 2: Univariate t-test for each LIWC variable by label
liwc_vars <- setdiff(predictors, c("unit_faculty", "demographic_sex"))

# Apply for-loop for clarity: run t-tests, then build the data frame
t_test_results <- lapply(liwc_vars, function(v) {
  tt <- t.test(train[[v]] ~ train$label)
  data.frame(variable = v,
             mean_irrel = tt$estimate[1],
             mean_rel = tt$estimate[2],
             t_stat = unname(tt$statistic),
             p_value = tt$p.value)
})
uni <- do.call(rbind, t_test_results)
uni$p_adj <- p.adjust(uni$p_value, method = "BH")
uni <- uni[order(abs(uni$t_stat), decreasing = TRUE), ]
head(uni, 15)
```

##		variable	mean_irrel	mean_rel
## mean in group Content-irrelevant		WC	36.3326203	160.2009685
## mean in group Content-irrelevant10		ppron	9.6484385	4.3666768
## mean in group Content-irrelevant5		WPS	12.6573476	22.5884140
## mean in group Content-irrelevant1		Analytic	54.5842246	79.3370339
## mean in group Content-irrelevant11		i	5.7613048	1.7116949
## mean in group Content-irrelevant9		pronoun	14.2488449	8.8300424
## mean in group Content-irrelevant6		Sixltr	18.5701070	26.0753692
## mean in group Content-irrelevant18		prep	10.0503850	13.6423245
## mean in group Content-irrelevant42		cause	1.4466738	3.0883051
## mean in group Content-irrelevant17		article	5.3663957	7.8849395
## mean in group Content-irrelevant57		affiliation	4.6146524	2.0607506
## mean in group Content-irrelevant63		focuspresent	13.1721711	9.1150242
## mean in group Content-irrelevant15		they	0.2122995	0.8063015
## mean in group Content-irrelevant72		money	0.3252193	1.3023245
## mean in group Content-irrelevant23		verb	16.8915294	12.6935412
##		t_stat	p_value	p_adj
## mean in group Content-irrelevant		-27.20340	1.891502e-138	1.759097e-136
## mean in group Content-irrelevant10		20.92911	2.248226e-83	6.969500e-82
## mean in group Content-irrelevant5		-20.82156	4.546200e-89	2.113983e-87
## mean in group Content-irrelevant1		-20.66984	7.331501e-83	1.704574e-81
## mean in group Content-irrelevant11		20.60563	1.744226e-80	3.244259e-79
## mean in group Content-irrelevant9		17.21877	7.434214e-60	1.152303e-58
## mean in group Content-irrelevant6		-16.30207	6.399095e-55	8.501655e-54
## mean in group Content-irrelevant18		-14.91951	1.078119e-46	1.114056e-45
## mean in group Content-irrelevant42		-14.82103	4.647752e-47	5.403012e-46
## mean in group Content-irrelevant17		-13.30423	2.505839e-38	2.330430e-37
## mean in group Content-irrelevant57		13.12147	9.802368e-37	8.287456e-36
## mean in group Content-irrelevant63		12.87095	1.409435e-35	1.008288e-34
## mean in group Content-irrelevant15		-12.71133	6.052196e-36	4.690452e-35
## mean in group Content-irrelevant72		-12.10064	8.714563e-33	5.788959e-32
## mean in group Content-irrelevant23		12.04949	1.143817e-31	7.091668e-31

```
# Compare overlap between top variables from both methods
intersect(names(head(imp, 15)), head(uni$variable, 15))
```

```
## [1] "WC"          "WPS"          "cause"        "i"            "ppron"
## [6] "pronoun"     "affiliation"
```

```
# Categorical predictors: chi-square tests
chisq.test(table(train$demographic_sex, train$label)) # Sex vs label
```

```
##
## Pearson's Chi-squared test
##
## data: table(train$demographic_sex, train$label)
## X-squared = 3.0287, df = 2, p-value = 0.22
```

```
chisq.test(table(train$unit_faculty, train$label)) # Faculty vs label
```

```
##
## Pearson's Chi-squared test
##
## data: table(train$unit_faculty, train$label)
## X-squared = 141.77, df = 9, p-value < 2.2e-16
```

```
# Proportion tables for interpretation
prop.table(table(train$demographic_sex, train$label), 1) # Relevance rate by sex
```

```
##
##          Content-irrelevant Content-relevant
## F          0.3551948      0.6448052
## M          0.3720038      0.6279962
## Unknown    0.0000000      1.0000000
```

```
prop.table(table(train$unit_faculty, train$label), 1) # Relevance rate by faculty
```

##		Content-irrelevant	Content-relevant
##	Faculty of Art Des & Architecture	0.2931507	0.7068493
##	Faculty of Arts	0.2264808	0.7735192
##	Faculty of Business & Economics	0.2037037	0.7962963
##	Faculty of Education	0.3992933	0.6007067
##	Faculty of Engineering	0.5357143	0.4642857
##	Faculty of Information Technology	0.4187726	0.5812274
##	Faculty of Law	0.5138889	0.4861111
##	Faculty of Med Nursing & Health Sci	0.2585366	0.7414634
##	Faculty of Pharmacy & Pharm Science	0.3535354	0.6464646
##	Faculty of Science	0.4611650	0.5388350

Two independent methods — decision-tree importance and a univariate t-test of each variable by class — agree on a core set of important variables (WC, WPS, cause, i, ppron, pronoun, affiliation). Word count is by far the strongest signal: content-relevant posts are much longer (160 vs 36 words) and more elaborated (WPS, Sixltr, Analytic).

Conversely, content-irrelevant posts use more first-person and personal pronouns and affiliation words, reflecting short social or logistical messages. In short, relevance is driven by post length and a formal-reasoning-versus-personal-chat contrast. This informs Q4, where I test whether a model built on this important subset can outperform Model 1.

The two categorical variables were tested with chi-square, the appropriate test for categorical predictors. Sex is not associated with the label ($\chi^2 = 3.03$, $p = 0.22$; relevance rate $\approx 64\%$ for female and 62.8% for male), whereas faculty is strongly associated ($\chi^2 = 141.8$, $p < 0.001$), with content-relevant rates ranging from about 46% (Engineering, Law) to 80% (Business & Economics, Arts).

Thus the important predictors are **post length and analytic-versus-personal language (numeric LIWC variables) together with the unit's faculty**; the author's sex carries no predictive value.

Because here is judging “relevant/irrelevant”, utilizing “class” method for `rpart()` (decision tree) function to predicts a discrete category or class label.

Question 4

Methods to be used:

- Decision tree (`rpart`)
- kNN
- Logistic regression (raw and log-transformed)

```
# Q4: Model comparison and improvement
```

```
library(rpart)
```

```
library(e1071)
```

```
library(class)
```

```
library(MLmetrics)
```

```
# 1. Isolate variable types
```

```
num_vars <- setdiff(names(train), c("Unique_ID", "unit_faculty", "demographic_sex", "label"))
```

```
all_vars <- setdiff(names(train), c("Unique_ID", "label"))
```

```
# 2. Select top 20 most important variables by t-statistic magnitude
```

```
get_t_stat <- function(variable_name) {
```

```
  test_result <- t.test(train[[variable_name]] ~ train$label)
```

```
  abs(unname(test_result$statistic))
```

```
}
```

```
tt_abs <- sapply(num_vars, get_t_stat)
```

```
important <- names(sort(tt_abs, decreasing = TRUE))[1:20]
```

```
# 3. Log-transform LIWC variables to address skewness
```

```
trainL <- train
```

```
valL <- val
```

```
trainL[num_vars] <- log1p(train[num_vars])
```

```
valL[num_vars] <- log1p(val[num_vars])
```

```
# 4. Initialize model leaderboard
```

```
cand <- list()
```

```
# Model 1: Baseline Logistic Regression (Raw Data)
```

```
m_base <- suppressWarnings(glm(reformulate(all_vars, "label"), data = train, family = binomial))
```

```
pred_base <- ifelse(predict(m_base, val, type = "response") > 0.5, "Content-relevant", "Content-irrelevant")
```

```
cand[["logistic all (Model 1)"]] <- F1_Score(y_true = val$label, y_pred = pred_base, positive = "Content-relevant")
```

```
# Model 2: Improved Logistic Regression (Log-Transformed Data)
```

```
m_log <- glm(reformulate(all_vars, "label"), data = trainL, family = binomial)
```

```
pred_log <- ifelse(predict(m_log, valL, type = "response") > 0.5, "Content-relevant", "Content-irrelevant")
```

```
cand[["logistic log1p all (improved)"]] <- F1_Score(y_true = val$label, y_pred = pred_log, positive = "Content-relevant")
```

```
# Model 3: Decision Tree
```

```
tree_model <- rpart(reformulate(all_vars, "label"), data = train, method = "class")
```

```
pred_tree <- predict(tree_model, val, type = "class")
```

```
cand[["decision tree all"]] <- F1_Score(y_true = val$label, y_pred = pred_tree, positive = "Content-relevant")
```

```
# Model 4: kNN using only the 20 'important' features
```

```
sc <- scale(train[num_vars])
```

```
va_sc <- scale(val[num_vars], center = attr(sc, "scaled:center"), scale = attr(sc, "scaled:scale"))
```

```
ed:scale"))
pred_knn <- knn(train = sc[, important], test = va_sc[, important], cl = train$label, k
= 9) # you can customize inition k-value
cand[["kNN important k=9"]] <- F1_Score(y_true = val$label, y_pred = pred_knn, positive
= "Content-relevant")

# 5. Print out the leaderboard
data.frame(model = names(cand), F1 = round(unlist(cand), 4))
```

```
##                                model      F1
## logistic all (Model 1)          logistic all (Model 1) 0.9171
## logistic log1p all (improved) logistic log1p all (improved) 0.9227
## decision tree all              decision tree all 0.9018
## kNN important k=9              kNN important k=9 0.9158
```

```
# 6. Final analysis of the winner (logistic regression on log-transformed data)
F1_Score(y_true = val$label, y_pred = pred_log, positive = "Content-relevant") # F1 val
idation score
```

```
## [1] 0.9226869
```

```
table(predicted = pred_log, actual = val$label) # Confusion matrix
```

```
##                actual
## predicted      Content-irrelevant Content-relevant
## Content-irrelevant          130           21
## Content-relevant            40           364
```

Question 5


```

# Q5: Predict on test set and prepare submission

# Final model: logistic regression on log-transformed variables
num_vars <- setdiff(names(train), c("Unique_ID", "unit_faculty", "demographic_sex", "label"))
all_vars <- setdiff(names(train), c("Unique_ID", "label"))

trainL <- train
trainL[num_vars] <- log1p(train[num_vars])
final_model <- glm(reformulate(all_vars, "label"), data = trainL, family = binomial)

# Read the test files (do NOT drop rows: every row needs a label)
ind_test <- read.csv("TaskD_Dataset/forum_ind_test.csv", stringsAsFactors = FALSE)
dep_test <- read.csv("TaskD_Dataset/forum_dep_test.csv", stringsAsFactors = FALSE)

# Clean up categoricals and apply same factor levels as training
ind_test$unit_faculty <- ifelse(is.na(ind_test$unit_faculty), "Unknown", ind_test$unit_faculty)
ind_test$demographic_sex <- ifelse(ind_test$demographic_sex %in% c("F", "M"), ind_test$demographic_sex, "Unknown")
ind_test$unit_faculty <- factor(ind_test$unit_faculty, levels = levels(train$unit_faculty))
ind_test$demographic_sex <- factor(ind_test$demographic_sex, levels = levels(train$demographic_sex))
ind_test[num_vars] <- log1p(ind_test[num_vars]) # Apply same transformation

# Predict and fill dep_test's label column in its original row order
prob <- predict(final_model, ind_test, type = "response")
pred <- ifelse(prob > 0.5, "Content-relevant", "Content-irrelevant")
pred_by_id <- data.frame(Unique_ID = ind_test$Unique_ID, pred = pred)
dep_test$label <- pred_by_id$pred[match(dep_test$Unique_ID, pred_by_id$Unique_ID)]

# Handle any test row with all-NA features by assigning the majority class
dep_test$label[is.na(dep_test$label)] <- names(sort(table(train$label), decreasing = TRUE))[1]

# Write submission: same 2 columns, 556 rows, unquoted
write.csv(dep_test, "Zhao_35507071_forum_label_test.csv", row.names = FALSE, quote = FALSE)

```